

Lab 3: Component Modelling & Architectural Pattern Selection

Problem Statement #44 — Database Query Performance Profiler

Name:Saurav Shashidhara

SRN:PES1UG24AM398

SEC:G

Scenario Recap

A database observability tool that ingests slow query logs, parses SQL execution plans, identifies missing index candidates, and generates weekly performance optimization digests. Stakeholders: Database Administrator, Backend Lead.

- Must parse Postgres/MySQL EXPLAIN plans and recommend index definitions (FR-001).
- Must ingest up to 5,000 slow-query log records/minute without dropping entries or slowing the host DB (NFR-001).

Written Justification (Required Deliverable)

Architecture Selection: "We chose Microservices Architecture for the Database Query Performance Profiler."

Architectural Choice

Microservices Architecture — the system is decomposed into independently deployable services aligned to the pipeline stages: log ingestion, plan parsing, index recommendation, digest reporting, and notification/dashboard delivery, backed by a shared metadata store.

Two Reasons

- Independent scaling: the Log Ingestion Service must sustain up to 5,000 records/minute continuously (NFR-001), while the Digest Report Generator only runs a heavy computation weekly. Microservices allow each service to be scaled and resourced independently instead of over-provisioning one monolithic tier for the ingestion peak at all times.
- Fault isolation: NFR-002-style reliability requires that log ingestion never silently drop data. If the Index Recommendation Engine or Digest Report Generator crashes or is redeployed, the Log Ingestion Service keeps accepting and durably queuing logs — a failure in one stage cannot cascade into data loss upstream, unlike in a tightly coupled layered design.

Security Advantage

Each microservice can be independently secured with least-privilege access: only the Log Ingestion Service needs direct network access to the production Postgres/MySQL log streams, while the Index Recommendation Engine, Digest Report Generator, and Notification & Dashboard Service communicate over an internal service boundary (authenticated APIs / message queue with mTLS) and never touch the production database directly. This limits the blast radius if any single service is compromised.

Performance Benefit

Decoupling the high-frequency ingestion path from the heavier index-analysis and weekly digest-generation stages means the 5,000-records/minute ingestion path is never blocked waiting on CPU-intensive plan analysis or report rendering — those run asynchronously off a queue, keeping ingestion latency low and predictable even under peak load.

Appendix A — Architectural Style Analysis

Style	Pros (for this scenario)	Cons (for this scenario)
Layered	Simple to build and reason about for a first version.	Tight coupling makes it hard to scale the high-throughput ingestion layer independently from the heavy, periodic digest-generation workload.
Microservices (chosen)	Ingestion, parsing, recommendation, and reporting scale and fail independently; matches the very different load shapes (continuous 5,000/min ingestion vs. weekly batch digest).	Higher operational complexity; network latency between services; eventual consistency across the pipeline.
Client-Server	Straightforward for the DBA/Backend Lead dashboard clients talking to a central service.	A single centralized server becomes a bottleneck / single point of failure under the sustained high-volume log ingestion requirement.

Appendix B — Component Diagram Summary

The full UML Component Diagram is provided as a separate draw.io file: DB_Query_Profiler_Component_Diagram.drawio. It contains 6 components and 7+ provided/required interface connections plus one <<use>> dependency (exceeding the minimum of 5 components / 4 interfaces):

- Log Ingestion Service — provides ILogStream
- Query Plan Parser Service — requires ILogStream & IDataAccess; provides IParsedPlan
- Index Recommendation Engine — requires IParsedPlan & IDataAccess; provides IRecommendation
- Digest Report Generator — requires IRecommendation & IDataAccess; provides IDigest
- Notification & Dashboard Service — requires IDigest; <<use>> dependency on Metadata & Query Log Store for historical trend look-ups
- Metadata & Query Log Store — provides IDataAccess to three consuming services

Actors Database Administrator and Backend Lead interact with the Notification & Dashboard Service to review digests and act on recommendations.

